

Введение

Эта статья - развитие [предыдущей](#), где рассматривалась реализация чат-бота с использованием Pub/Sub. В этот раз сосредоточимся на создании мультиплатформенного бота. Под платформой подразумевается любой канал связи, например, мессенджер или виджет на сайте. Это востребованное решение: один бот, доступный на нескольких платформах, увеличивает охват потенциальных пользователей.

Основной акцент будет сделан на разработке архитектуры, которая позволяет легко адаптировать бота к новым платформам. В этой статье реализуем чат-бота, работающего через Telegram и WebSocket, с возможностью дальнейшего расширения.

Практическая часть

Архитектура

Архитектура бота основывается на принципах модульности и масштабируемости:

1. **Core Bot** — центральный компонент, отвечающий за логику обработки сообщений. Он полностью независим от платформ.
2. **Сервисы для взаимодействия с пользователем:**
 - **Telegram Bot:** для общения через Telegram.
 - **WebSocket Chat:** для взаимодействия через WebSocket.

Для взаимодействия между сервисами используется Redis Pub/Sub. Это решение обеспечивает:

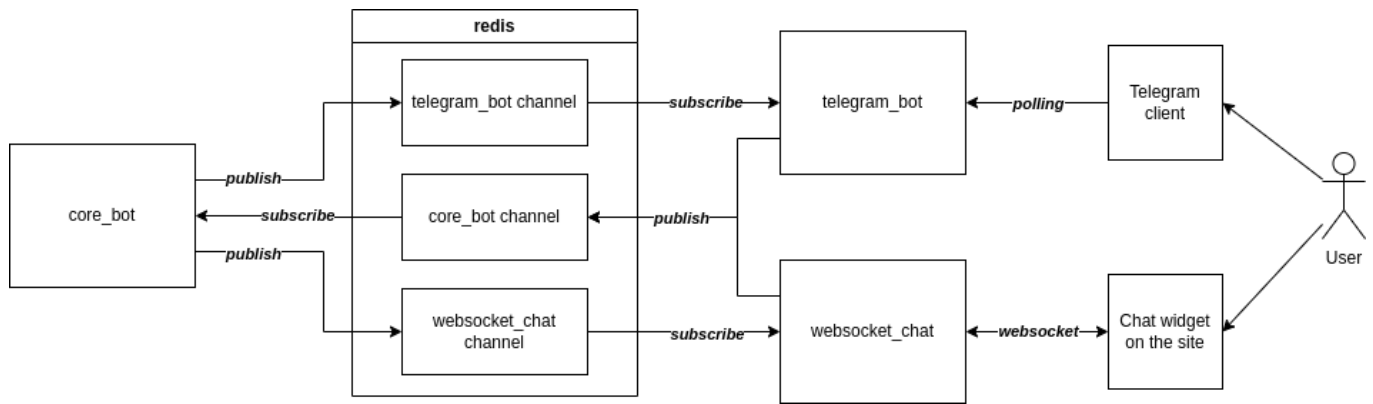
- Независимость сервисов.
- Масштабируемость (возможность разделения нагрузки).
- Простоту добавления новых платформ.

Общий принцип работы:

1. Сообщение от пользователя передаётся в Core Bot через соответствующий канал Redis.
2. Core Bot обрабатывает сообщение и отправляет ответ обратно через указанный канал.

Таким образом, Redis обеспечивает двунаправленную связь между сервисами.

Упрощенная схема взаимодействия выглядит так:



Развёртывание системы

Для запуска компонентов используется `docker-compose`, описывающий 4 контейнера:

```

services:
  telegram_bot:
    build:
      context: .
      dockerfile: ci/telegram_bot/Dockerfile
    networks:
      - chat
    environment:
      - TELEGRAM_BOT_TOKEN={{ your-token }}

  websocket_chat:
    build:
      context: .
      dockerfile: ci/websocket_chat/Dockerfile
    networks:
      - chat
    ports:
      - "8000:8000"

  core_bot:
    build:
      context: .
      dockerfile: ci/core_bot/Dockerfile
    networks:
      - chat

  redis:
    image: redis:7.4
    networks:
      - chat
    ports:

```

```
- "6379:6379"
volumes:
  - redis-data:/data
command: ["redis-server"]

networks:
  chat:
    driver: bridge

volumes:
  redis-data:
    external: true
```

При этом описание всех Dockerfile примерно одинаковое:

```
FROM python:3.12-slim

# Установка зависимостей
COPY ci/core_bot/requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

# Копирование исходного кода
WORKDIR /src
COPY src/core_bot .
COPY src/pub_sub.py ./pub_sub.py

ENTRYPOINT [ "python3", "run.py" ]
```

В представленном Dockerfile:

1. В качестве базового образа берём официальный образ [Python](#);
2. Устанавливаем зависимости с помощью менеджера пакетов `pip`;
3. Копируем кодовую базу;
4. Задаём точку входа в контейнер;

Реализация

После того как определились с развёртыванием, приступаем к реализации! Весь дальнейший код представлен в репозитории - <https://github.com/anatoly-bobrovsky/multi-platform-chatbot>.

Для начала определим необходимые в дальнейшем зависимости для каждого контейнера:

1. **core_bot**:

```
redis==5.2.1 # pub/sub
```

2. **telegram_bot**:

```
aiogram==3.15.0 # telegram bot  
pydantic-settings==2.7.0 # for env  
redis==5.2.1 # pub/sub
```

3. **websocket_chat**:

```
fastapi==0.115.6 # API  
redis==5.2.1 # pub/sub  
uvicorn==0.33.0 # ASGI web server  
websockets==14.1 # websocket chats
```

Redis Pub/Sub

Основная логика работы через Redis вынесена в модуль `src/pub_sub.py`. Он содержит:

- Функцию публикации (`publish`), которая отправляет сообщения в указанный канал.
- Функцию подписки (`subscribe`), обрабатывающую входящие сообщения.

```
"""Redis Pub/Sub."""  
  
import json  
from dataclasses import asdict, dataclass  
from enum import Enum  
from typing import Awaitable, Callable  
  
import redis.asyncio as redis  
  
class Channel(Enum):  
    """Enumeration of available channels for Redis Pub/Sub."""  
  
    TELEGRAM_BOT = "telegram_bot"  
    CORE_BOT = "core_bot"  
    WEBSOCKET_CHAT = "websocket_chat"
```

```

@dataclass
class Message:
    """Data class representing a message to be sent through Redis Pub/Sub.

    Attributes:
        text (str): The text content of the message.
        chat_id (str): The identifier for the chat.
        callback_channel (Channel): The channel to which the message should
be sent.
    """

    text: str
    chat_id: str
    callback_channel: Channel

    def __str__(self) -> str:
        """Convert the message to a JSON string.

        Returns:
            str: JSON string representation of the message.
        """
        message_dict = asdict(self)
        message_dict["callback_channel"] = self.callback_channel.value
        return json.dumps(message_dict)

redis_client = redis.from_url("redis://redis:6379")

async def subscribe(channel: Channel, process_message: Callable[[Message],
Awaitable[None]]) -> None:
    """Subscribe to a channel and process incoming messages.

    Args:
        channel (Channel): The channel to subscribe to.
        process_message (Callable[[Message], Awaitable[None]]): The function
to process incoming messages.
    """
    async with redis_client.pubsub() as pubsub:
        await pubsub.subscribe(channel.value)

        while True:
            redis_message: dict = await
pubsub.get_message(ignore_subscribe_messages=True)
            if redis_message is not None:

```

```

        message_dict = json.loads(redis_message["data"])
        message_dict["callback_channel"] =
Channel(message_dict["callback_channel"])
        await process_message(Message(**message_dict))

async def publish(channel: Channel, message: Message) -> None:
    """Publish a message to a channel.

    Args:
        channel (Channel): The channel to publish the message to.
        message (Message): The message to be published.
    """
    await redis_client.publish(channel.value, str(message))

```

Как было указано в схеме раньше, есть 3 канала на каждый сервис: `TELEGRAM_BOT`, `CORE_BOT` и `WEBSOCKET_CHAT`. Обмен будет происходить сообщениями, логика сериализации которых, представлена в датаклассе `Message`. После получения сообщения сервис будет обрабатывать его по своему усмотрению (асинхронная функция `process_message`). Также стоит отметить, что в `Message` выделен отдельный атрибут `callback_channel`, который отвечает за то, в какой сервис нужно вернуть ответ со стороны `core_bot`.

WebSocket Chat

В `src/websocket_chat/api_app.py` реализован WebSocket-сервер на FastAPI, который обрабатывает входящие сообщения и отправляет их в `CORE_BOT` канал. Ответы из `WEBSOCKET_CHAT` канала отправляются обратно клиенту.

```

@app.websocket("/chatbot/{chat_id}")
async def websocket_endpoint(websocket: WebSocket, chat_id: str) -> None:
    """
    WebSocket endpoint for chatbot communication.

    Args:
        websocket (WebSocket): The WebSocket connection object.
        chat_id (str): The unique identifier for the chat session.

    This function handles WebSocket connections for a chatbot. It accepts
    the WebSocket connection, then runs two asynchronous tasks concurrently:
    one for receiving messages from the client and another for sending
    messages to the client.
    """
    await websocket.accept()

```

```

await run_until_first_complete(
    (chat_ws_receiver, {"websocket": websocket, "chat_id": chat_id}),
    (chat_ws_sender, {"websocket": websocket, "chat_id": chat_id}),
)

```

В функциях `chat_ws_receiver` и `chat_ws_sender` используются функции публикации и подписки, которые определены выше.

Telegram Bot

С телеграм ботом посложнее: надо запустить две асинхронные задачи - одна будет подписываться на `TELEGRAM_BOT` канал, а вторая будет поллить API Телеграма. Таким образом, в `src/telegram_bot/run.py` происходит инициализация телеграм бота:

```

async def main() -> None:
    """
    Main function to initialize and run the Telegram bot.

    This function sets up the bot with the token and default properties,
    defines a message processing function, and starts both the message
    subscription and the polling for updates.
    """
    bot = Bot(token=env.TELEGRAM_BOT_TOKEN,
default=DefaultBotProperties(parse_mode=ParseMode.HTML))

    async def process_message(message: Message) -> None:
        """
        Process a message by sending it to the appropriate chat.

        Args:
            message (Message): The message object containing chat_id and
text.
        """
        await bot.send_message(chat_id=int(message.chat_id),
text=message.text)

    await asyncio.gather(
        subscribe(channel=Channel.TELEGRAM_BOT,
process_message=process_message), dp.start_polling(bot)
    )

```

На самих хендлерах телеграм бота будет публикация сообщений в `CORE_BOT` канал:

```

@dp.message(CommandStart())
async def command_start_handler(message: TelegramMessage) -> None:
    """
    Handles the /start command.

    Args:
        message (TelegramMessage): The message object containing the
        command.
    """
    await message.answer("Hello!")

@dp.message()
async def message_handler(message: TelegramMessage) -> None:
    """
    Handles all incoming messages and publishes them to the core bot
    channel.

    Args:
        message (TelegramMessage): The message object containing the text
        and chat information.
    """
    await publish(channel=Channel.CORE_BOT,
message=Message(text=message.text, chat_id=str(message.chat.id),
callback_channel=Channel.TELEGRAM_BOT))

```

Core Bot

И наконец последний сервис, который отвечает за обработку сообщений. Для примера в `src/core_bot/run.py` реализован возврат эхо-ответа:

```

async def main() -> None:
    async def process_message(message: Message) -> None:
        await publish(channel=message.callback_channel,
message=get_echo_response(message))
    await subscribe(channel=Channel.CORE_BOT,
process_message=process_message)

```

Но в вашем случае это может быть сколь угодно сложная логика обработки, например, использование цепи LLM агентов.

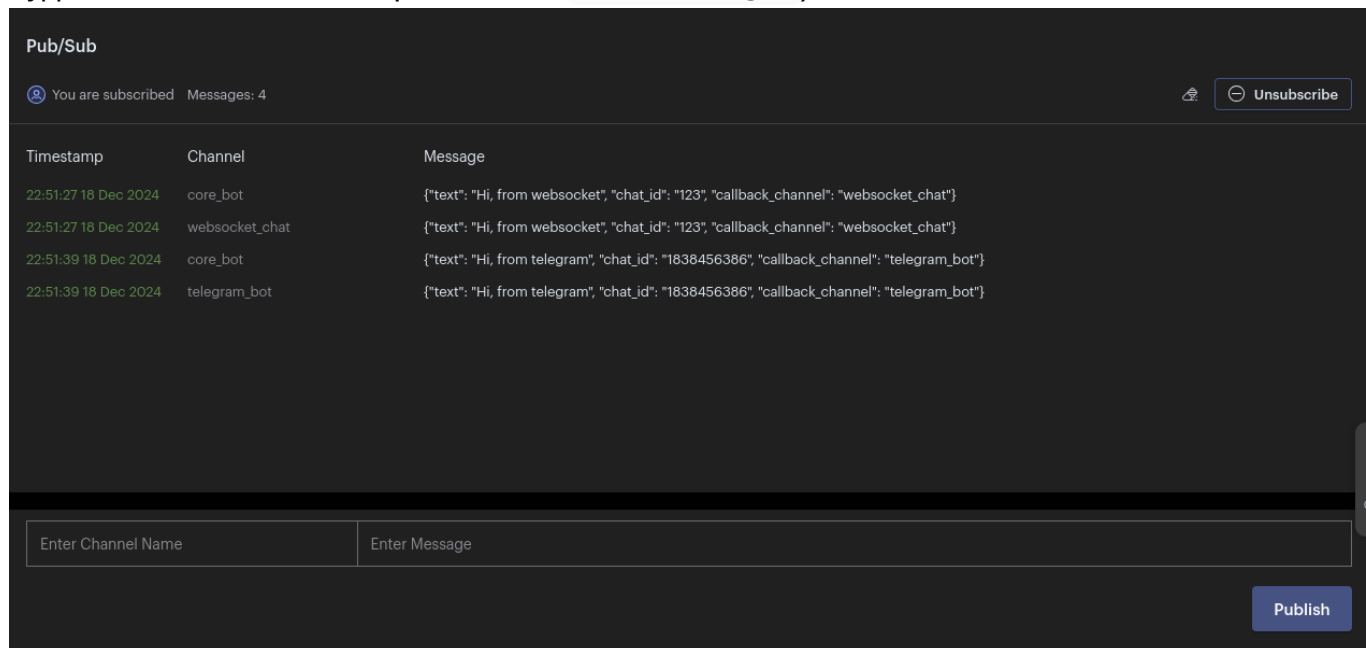
Запуск бота

Давайте попробуем запустить нашего эхо-бота! Для этого:

1. Создаем внешний вольюм: `docker create volume redis-data;`
2. Проставляем в `docker-compose.yml` переменную среды `TELEGRAM_BOT_TOKEN` согласно вашему токену бота;
3. Запускаем: `docker compose up -d --build;`

Websocket чат можно протестировать с помощью `Postman`, как и в прошлой статье, а Telegram бота - через любой клиент Telegram. В обоих случаях бот будет присылать такое же сообщение в ответ. Таким образом, одна логика взаимодействия переносится на несколько платформ!

Чтобы убедиться, что всё работает правильно, заглянем в `Redis` (как и в прошлый раз будет использовать GUI приложение `Redis Insight`):



Как видно из скриншота, сообщения летят в правильные каналы!

Заключение

В этой статье разработали и развернули мультиплатформенного чат-бота с помощью Python, Docker и Redis Pub/Sub. Такой подход позволяет легко расширять функциональность бота, добавляя новые платформы - для этого нужно описать сервис обработки сообщений для вашей новой платформы и создать канал в Redis для публикации сообщений!